

# Relatório: Trabalho 3 – Web Services (WS) e Application Interface Programming (API)

**Disciplina:** Sistemas Distribuídos

**Instituição:** UFC - Campus Quixadá

**Autoras:**

- Gabriely Correia Dealem
  - Laura Soléria Lobo Vitorino Maciel
- 

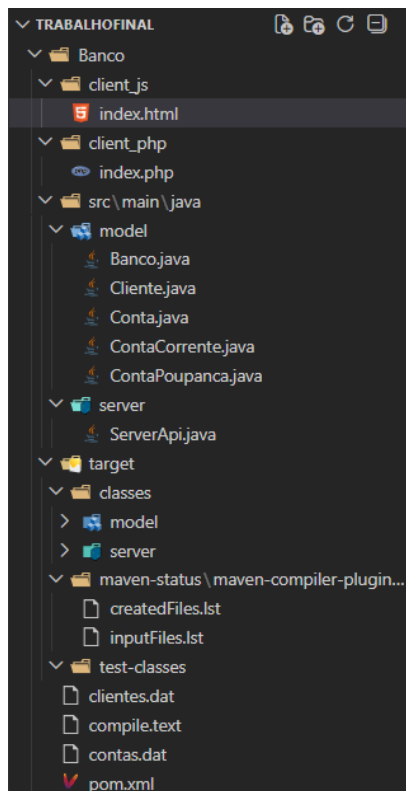
## 1. Introdução e Objetivo

Este relatório descreve o desenvolvimento de um sistema bancário distribuído utilizando o conceito de **Web Services** e **API REST**. O objetivo principal foi implementar a interoperabilidade entre diferentes linguagens de programação. Para isso, foi criado um servidor centralizado em Java, que expõe métodos de negócio (criação de conta, depósito, saque, transferência e consulta de saldo) através do protocolo HTTP, permitindo que clientes desenvolvidos em PHP e JavaScript consumam esses serviços de forma transparente e distribuída.

## 2. Arquitetura do Sistema

A solução segue o modelo **Cliente-Servidor** baseada em requisições HTTP, caracterizando uma arquitetura RESTful (ou REST-like):

- **Servidor (Back-end - Pasta Banco):** Desenvolvido em Java. Atua como o provedor do Web Service, escutando uma porta específica, processando requisições HTTP (GET/POST) e retornando respostas (geralmente em formato JSON ou texto).
- **Cliente PHP (Pasta client\_php):** Aplicação Server-side que atua como consumidor da API, realizando requisições HTTP ao servidor Java para processar dados.
- **Cliente JS (Pasta client\_js):** Aplicação Client-side (SPA) que roda no navegador, utilizando fetch ou AJAX para interagir dinamicamente com a API Java.
- **Comunicação:** O sistema utiliza o protocolo **HTTP** como camada de transporte e **JSON** (JavaScript Object Notation) como formato de intercâmbio de dados, garantindo que o Java, PHP e JS se compreendam.



### 3. Implementação do WebService (Java)

O núcleo do sistema (Back-end) foi construído para ser o detentor do "Estado" da aplicação e expor a API:

- **Modelo de Dados:** Classes Java (Conta, Cliente) que representam as entidades bancárias.
- **Persistência:** Uso do arquivo `clientes.dat` (serialização de objetos ou escrita direta) para garantir a persistência dos dados entre reinicializações do servidor.
- **Endpoints (Rotas):** O servidor intercepta requisições HTTP e as roteia para os métodos adequados. Exemplos lógicos da API:
  - `POST /criar`: Recebe dados e instancia uma nova conta.
  - `POST /saque` e `POST /deposito`: Recebe ID da conta e valor para atualizar o saldo.
  - `GET /saldo`: Recebe ID da conta e retorna o valor atual.
- **Tratamento de Concorrência:** Implementação de `synchronized` nos métodos críticos de manipulação de saldo para evitar condições de corrida quando múltiplos clientes acessam a mesma conta simultaneamente.

### 4. Implementação dos Clientes (PHP e JS)

#### Cliente PHP

O cliente PHP demonstra a integração Back-end to Back-end.

- Utiliza bibliotecas nativas (como cURL ou `file_get_contents`) para enviar requisições HTTP ao servidor Java.
- Processa a resposta JSON vinda do Java e renderiza o HTML final para o usuário.
- Funciona como um intermediário: O usuário fala com o PHP, e o PHP fala com o Java.

```

1  <?php      Gab-Dealem04, yesterday • Subindo projeto sistema bancario
2  session_start(); // Inicia a memória do navegador
3
4  $mensagem = '';
5  $tipoMensagem = '';
6  // Endereço do servidor Backend (API Java) onde os dados reais são processados e armazenados
7  $apiUrl = "http://172.30.112.60:7000";
8
9  // --- LÓGICA DE SAIR (LOGOUT) ---
10 // Verifica se a URL tem o parâmetro ?sair=true. Se tiver, limpa a sessão e desloga.
11 if (isset($_GET['sair'])) {
12     session_destroy();
13     header("Location: index.php");
14     exit;
15 }
16
17 // --- PROCESSAMENTO DOS FORMULÁRIOS ---
18 // Só entra aqui se o usuário clicar em algum botão de envio (POST)
19 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
20
21     // 1. LÓGICA PARA CRIAR CONTA E JÁ ENTRAR
22     if (isset($_POST['acao']) && $_POST['acao'] == 'criar') {
23         $url = $apiUrl . "/contas";
24         // Prepara o pacote de dados (Array) para enviar via JSON para a API
25         $data = [
26             "tipo" => $_POST['tipo'],
27             "numero" => $_POST['numero'],
28             "nome" => $_POST['nome'],
29             "cpf" => $_POST['cpf'],
30             "idade" => $_POST['idade'],
31             "extra" => $_POST['extra']
32         ];
33         // Chama a função auxiliar para fazer o POST no Java
34         $res = enviarRequisicao($url, 'POST', $data);
35
36         // Código 201 significa "Created" (Criado com sucesso) no protocolo HTTP
37         if ($res['codigo'] == 201) {
38             // Se criou com sucesso, loga o usuário automaticamente
39             $_SESSION['conta_logada'] = $_POST['numero'];
40             $_SESSION['nome_usuario'] = $_POST['nome']; // Guarda o nome pra ficar bonito
41             $mensagem = "Conta criada! Bem-vindo.";
42             $tipoMensagem = 'sucesso';
43         } else {
44             $mensagem = "Erro ao criar: " . $res['corpo'];
45             $tipoMensagem = 'erro';
46         }
47     }

```

## Cliente JavaScript

Focado na experiência do usuário e interatividade (Front-end).

- Utiliza a API `fetch()` do navegador para fazer chamadas assíncronas ao servidor Java.

- Manipula o DOM para atualizar o saldo na tela instantaneamente sem recarregar a página (AJAX).
- Lida com respostas de erro (ex: "Saldo insuficiente") exibindo alertas diretamente na interface.

```

91 <script>
92 // Configuração do endereço do servidor Backend (API Java)
93 const API_URL = "http://172.30.112.60:7000";
94
95 // ESTADO DA APLICAÇÃO: Diferente do PHP (sessão no servidor),
96 // aqui a "sessão" vive na memória RAM do navegador enquanto a página estiver aberta.
97 let contaLogada = null; // Variável Global que segura a "Sessão"
98
99 // --- FUNÇÕES DE NAVEGAÇÃO (TELA 1 <-> TELA 2) ---
100
101 // Gerencia a troca de telas sem recarregar a página (Manipulação de DOM).
102 function mudarTela(logado) {
103     if (logado) {
104         // Esconde a tela de login e mostra a tela principal
105         document.getElementById('tela-login').classList.add('hidden');
106         document.getElementById('tela-dashboard').classList.remove('hidden');
107         document.getElementById('lbl-conta').innerText = contaLogada;
108         document.getElementById('output').innerText = "Bem-vindo! Selecione uma opção.";
109     } else {
110         // Processo inverso para o Logout
111         document.getElementById('tela-login').classList.remove('hidden');
112         document.getElementById('tela-dashboard').classList.add('hidden');
113         contaLogada = null;
114         // Limpa campos
115         document.getElementById('op_valor').value = "";
116         document.getElementById('output').innerText = "";
117     }
118 }
119
120 function fazerLogin() {
121     const num = document.getElementById('login_numero').value;
122     if (num) {
123         contaLogada = num; // Salva o número da conta na variável global
124         mudarTela(true); // Vai para o Dashboard
125     } else {
126         alert("Digite um número de conta!");
127     }
128 }
129
130 function fazerLogout() {
131     mudarTela(false); // Volta para o Login
132 }
133
134 // --- FUNÇÕES DE API (COMUNICAÇÃO COM O JAVA) ---
135 // Captura os dados do formulário de cadastro e envia via POST (JSON) para a API.

```

## 5. Tecnologias Utilizadas

- **Linguagens:** Java (Servidor), PHP e JavaScript (Clientes).
- **Protocolo de Comunicação:** HTTP (Hypertext Transfer Protocol).
- **Intercâmbio de Dados:** JSON.
- **Back-end:** Java Standard Edition (Socket Server ou Framework HTTP).
- **Armazenamento:** Arquivo binário sequencial (.dat).
- **Gerenciamento de Build:** Maven (conforme estrutura pom.xml implícita nos comandos).

## 6. Execução do Sistema

## 6.1. Inicialização do Servidor (Java)

No diretório `TrabalhoFinal/Banco`, compilar e executar a classe principal que sobe o serviço HTTP:

Bash

```
mvn clean compile exec:java -Dexec.mainClass="server.ServerApi"
```

*(O servidor ficará aguardando requisições na porta configurada, ex: 8080)*

## 6.2. Execução dos Clientes

- **PHP:** Iniciar o servidor embutido na pasta `client_php`:

Bash

```
php -S localhost:8000
```

Acessar via navegador em <http://localhost:8000>.



**Bem-vindo ao Banco**

Número da Nova Conta

Seu Nome

CPF

Idade

Conta Corrente

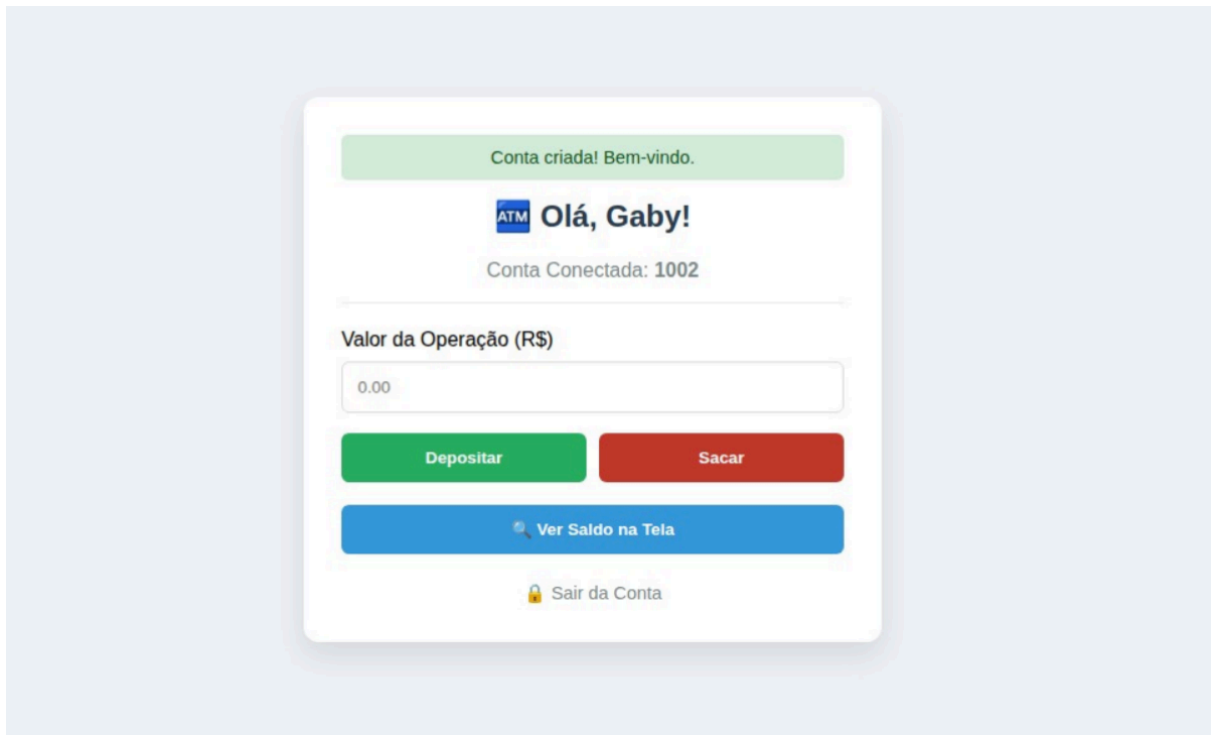
Limite ou Variação

**Criar Conta e Entrar**

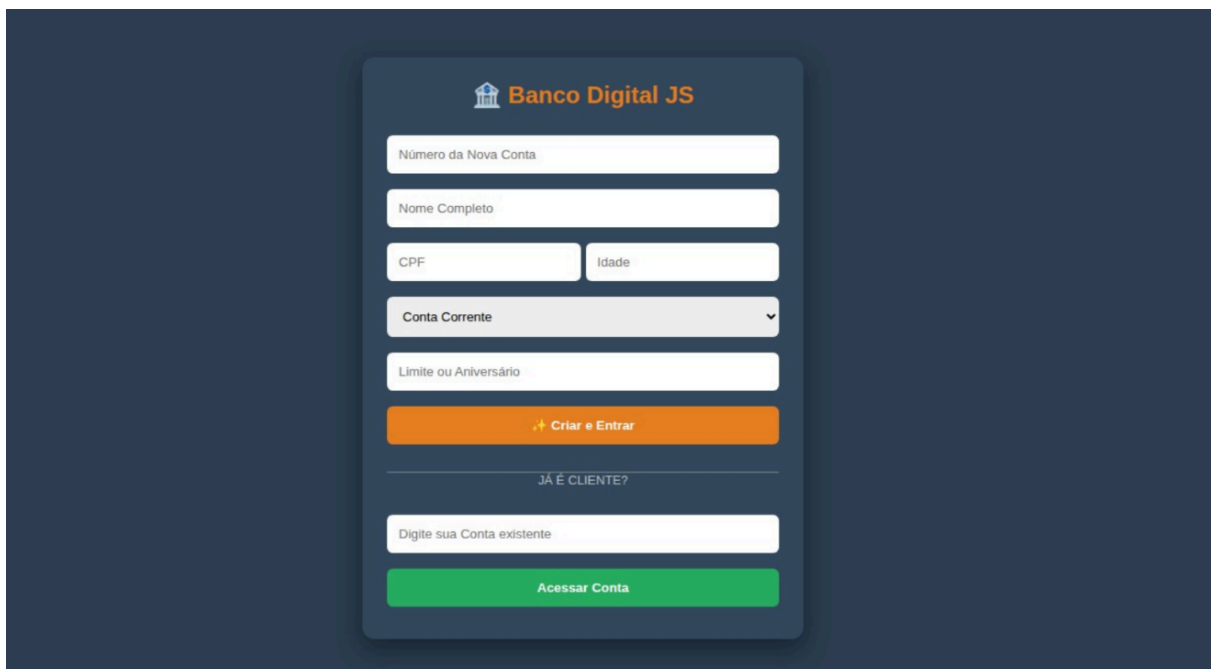
JÁ TEM CONTA? ACESSE ABAIXO

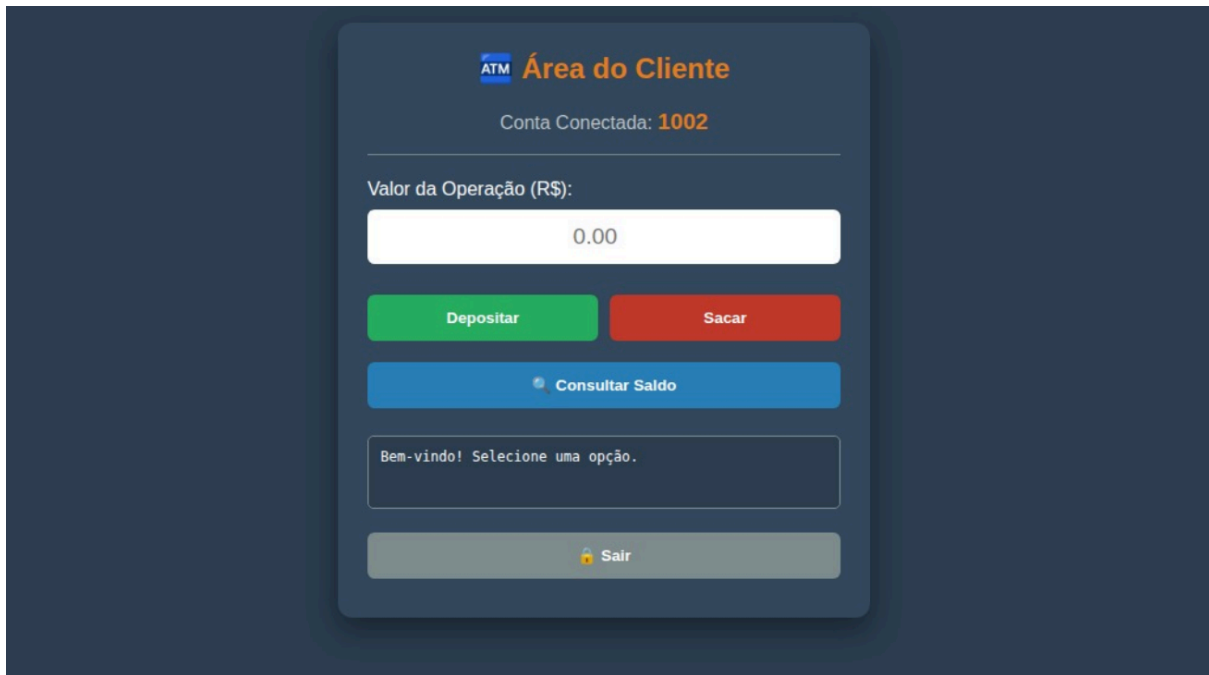
Número da Conta Existente

**Acessar Conta**



- **JavaScript:** Abrir o arquivo `index.html` da pasta `client_js` diretamente no navegador ou via Live Server.





---

## 7. Desafios e Resolução de Problemas

1. **Interoperabilidade:** O maior desafio foi garantir que os dados enviados pelo PHP/JS fossem corretamente "parseados" pelo Java e vice-versa. A padronização do formato JSON foi a solução adotada.
2. **CORS (Cross-Origin Resource Sharing):** No cliente JavaScript, o navegador bloqueava requisições para o servidor Java por estarem em portas diferentes. Foi necessário adicionar cabeçalhos HTTP no servidor Java (`Access-Control-Allow-Origin: *`) para permitir a conexão.
3. **Persistência Concorrente:** A manipulação do arquivo `clientes.dat` exigiu cuidado com Threads para evitar corrupção de dados durante acessos simultâneos de escrita.

## 8. Conclusão

A atividade demonstrou com sucesso a aplicação prática de Web Services. A arquitetura desacoplada permitiu que a regra de negócio permanecesse centralizada no Java,

enquanto diferentes interfaces (Web PHP e Web JS) consumissem os mesmos dados. Isso reflete o cenário real de microsserviços e APIs, onde a linguagem do servidor é indiferente para quem consome o serviço, desde que o protocolo (HTTP) e o formato de dados (JSON) sejam respeitados.